Agent 長期記憶系統

框架對比、認知架構與選型決策 | v3 | 2026-05-02

報告日期：2026-05-02
撰寫方：hv-research / GenAI Frontiers

目錄

1. **Agent** 長期記憶系統 **v3**

30 秒選型卡 + 11 個選項抽象層級 + 各章本節結論

Agent 長期記憶系統：框架對比、認知架構與選型決策

> 橫縱分析法深度研究報告 ｜ v3 ｜ 2026-04-29 ｜ 資料截止：2026-04-29

30 秒選型卡：直接告訴我該選哪個

> 如果你只有 **30** 秒，看完這張卡就走——後續章節是論證與細節，不看不影響你做決定。

你的情境	第一推薦	第二選擇	為什麼（一句話）
Claude 生態開發者（Code / Agent SDK）	**Anthropic Memory tool** + CLAUDE.md	+ Mem0（補多 user）	tool primitive 路線、不需 external service、與 Claude 工作流原生對齊
Self-host 企業（資料不能離內網）	**Mem0 OSS** 或 **Zep self-host**	Cognee（air-gapped）/ Letta self-hosted	Long Context / 託管 memory 都不可用，唯一路徑就是 framework
短歷史 chatbot（< 500K tokens）	**Long Context + Prompt Cache**	Anthropic Memory tool	不需框架就解決，便宜 10×、簡單 10×
同 project coding session	**Long Context**（codebase 全塞）	CLAUDE.md + Memory tool	session 內單純，不值得上 memory pipeline
跨月跨年個人偏好累積	**Mem0**	LangMem（若已在 LangGraph）	extraction-heavy 但 read 飛快、生態整合最廣
事實會變的長期 agent（客服 / 法律 / 醫療）	**Zep / Graphiti**	Mem0g	唯一原生支援 temporal fact validity window
跨任務累積技能的 autonomous agent	**Letta**	—	唯一原生支援「agent 永遠在跑」的 OS-style 模型
個人 PKM / 開發者 second brain	**Karpathy LLM Wiki** 模式	Anthropic Memory tool / Obsidian + Mem0	編譯一次 N 次受惠、cross-references 已就位
離線 / 隱私敏感（air-gapped, GDPR）	**MemPalace** 或 **Cognee**	+ Ollama 本地推論	verbatim 守原文、純 local、MIT 授權
ChatGPT C 端使用者	**OpenAI Memory**（內建）	—	不需 external 配置、產品層面已透明化

選型前的三個 **gate**：

1. **Session 累積會超過 500K tokens 嗎？** 不會 → 先試 Long Context + Cache，不要急著上 framework
2. **資料能離開內網嗎？** 不能 → 跳過所有閉源 API 與託管 memory，只看 self-host 路線
3. **你 commit 哪個 LLM 生態？** 單押 Claude → Anthropic Memory tool；多 provider → Mem0 / Zep / Letta

本報告閱讀導引

報告共 11 章，依不同身份提供建議路徑：

你的角色 / 目的	建議閱讀路徑	預估時間
趕時間決策者	**30** 秒選型卡 + **§0** 摘要	3 分鐘
想知道每個框架定位	**§2** 五大流派 **+** 三條對照路線 **+ LangMem**（11 個選項全在這）	15 分鐘
想了解設計取捨的根	**§3** 設計取捨 + **§5** 認知架構視角	25 分鐘
要做 benchmark 評估	**§4 Benchmark** 解讀	10 分鐘
主管要看選型結論	**§0** + **§7** 選型決策樹（兩層 gate + 場景速查）	10 分鐘
採購 / 投資判斷	**§7** + **§10** 能力限制聲明	15 分鐘
完整深度研究	全文按序	90 分鐘

每章開頭都有粗體「本節結論」3-5 行——只看本節結論就能掌握該章重點，細節留給有興趣的讀者。

---

§0 摘要：60 秒讀完

2026 年是 agent 元年的喧囂背後，最大的工程難題不是 reasoning、不是 tool use、是記憶。Karpathy 把這個現象命名為「**Decade of Agents**」——當前 LLM 像得了 anterograde amnesia 的天才，每次對話結束就忘光，所有持續性都靠工程外掛硬撐。

這份報告盤點當前主流的五大流派 **+** 三條對照路線，再把第三方 memory 框架攤在兩個維度上比較：

```
                Hierarchical / Graph
                        ▲
                        |
          Letta •       |     • Zep / Graphiti
        (OS paging)     |   (Temporal KG)
                        |
                        |     • Karpathy LLM Wiki
                        |     (Compiled markdown)
———————————————————————|————————————————————►
                        |              Extraction
    Verbatim            |
                        |     • Mem0
          MemPalace•    |     (LLM extraction + hybrid store)
        (Method of      |
          Loci)         |
                        ▼
                    Flat store
```

五大流派的核心取捨：

框架	核心隱喻	**Write** 成本	**Retrieval**	**LongMemEval**	何時用
Mem0	LLM extraction + hybrid	高（每次 LLM call）	快	49.0%	個人化 chatbot / B2B copilot
Zep / Graphiti	Temporal Knowledge Graph	中（自動 KG）	快、無 LLM	**63.8%**	事實會變的長期 agent
Letta（前 **MemGPT**）	OS paging（RAM/Disk）	中	LLM 自決	無公開	長 horizon autonomous
MemPalace	Method of Loci 空間隱喻	零 **LLM**	階層喚醒	96.6%（爭議大）/ 88.9%（獨立重現）	離線 / 隱私敏感
Karpathy LLM Wiki	Compiler（編譯一次）	中（人 + LLM 協作）	markdown 直查	無 benchmark	個人 PKM / 開發者 second brain

三條對照路線（不是 memory 框架本身，但會壓縮或重劃 memory 框架的市場空間）：

路線	核心主張	主要受限	何時優於第三方 **memory** 框架
Long Context + Prompt Cache（§2.6）	1M tokens + 5min cache，不需 memory framework	閉源 API only（Claude Sonnet 4.5 / Gemini 2.5 Pro / GPT-4.1）；自部署開源模型 1M 可行性差	短歷史 chatbot / 單 project coding session（< 500K tokens）
Anthropic Memory tool + Claude Skills（§2.7）	memory 是 LLM 的 tool primitive，不是獨立 service	與 Anthropic 生態綁定；非通用協議	Claude Code / Claude Agent SDK 開發者預設選項
OpenAI Memory / ChatGPT Personalization（§2.8）	託管於 ChatGPT 帳號層級，使用者透明化	黑箱、不可移植、非 API-first	C 端使用者；不適合 enterprise self-host

圖 0：11 個選項的抽象層級全景——選錯抽象層比選錯框架更貴。第零層適合「不需要記憶」的場景，第一層綁 Anthropic / OpenAI 生態，第二層是 multi-provider / self-host 主流，第三層是設計指引而非 product。

最值得深思的兩條深層張力：

張力一 **— Karpathy** 的三顆思想炸彈：

1. **System Prompt Learning** ——LLM 缺一個學習範式，不是 pretraining 也不是 fine-tune，而是「會自己寫筆記、總結教訓」
2. **LLM Wiki** ——這個範式的具體實作：對話 → daily log → 編譯成 wiki → 注入下次 session
3. **Cognitive Core** ——最終形態：瘦核心模型 + 外掛記憶，「I want to remove the memory... and only maintain the algorithms for thought」

張力二 **— Service-layer vs Tool-primitive** 的設計哲學衝突：

- **Service-layer** 派（Mem0 / Zep / Letta）：把 memory 當獨立服務，外部包圍 LLM
- **Tool-primitive** 派（Anthropic）：把 memory 當 LLM 的 tool primitive，由 LLM 自己讀寫
- 這是兩條不相容的路線，選錯框架會卡在錯的抽象層
- 企業 **self-host** 場景的真實限制：Long Context + Cache 只適用閉源 API；自部署開源模型（Qwen2.5-1M / GLM-4-1M）有 quality cliff 與算力門檻，因此 **regulated / sovereign / self-hosted** 場景仍必須依賴 memory 框架

如果 Karpathy 對了，現在這五家全是過渡形態，真正的記憶最終要烤回權重（§5.4 + §9.9）。但即使 Karpathy 對了一半，§2.6-§2.8 的三條路也會把第三方 memory 框架的市場壓縮到「self-host enterprise + 多 agent autonomous」這兩個 niche——這是 §7 選型決策樹的核心 reframe。

---

§1 為什麼 agent 需要「真正的記憶」

> 本節結論
> - 記憶是 2026 年 agent 真正卡住的瓶頸，不是 reasoning / tool use
> - 當前 LLM 像 anterograde amnesia 患者：權重訓練後就凍結，新對話內容無法沉澱

> - Karpathy 把 agent 元年改稱「Decade of Agents」，其中記憶是最尖銳的架構性缺陷
> - 業界四種解法（context window / Long Context+Cache / RAG / persistent memory）只有最後一種真正回答跨 session 連續性
> - 後續章節討論的 11 個選項都在做「persistent memory」這條長條的不同切法

§1.1 Anterograde Amnesia——Karpathy 的隱喻

順向失憶症（anterograde amnesia）是一種神經學病症：患者保有過去的記憶，但無法形成新的長期記憶。每天醒來，昨天發生的事都歸零。1985 年的英國音樂家 Clive Wearing 是最有名的個案，他每隔幾分鐘就「醒過來」一次，每次都以為自己剛從昏迷中復甦，認得出妻子但記不住她下午來看過。

Karpathy 用這個病作為當前 LLM 的隱喻。他在 2025 年 10 月接受 Dwarkesh Patel 訪談時說了一段被廣泛轉貼的話：

> "They don't have continual learning. You can't just tell them something and they'll remember it."

訓練結束後，模型權重就凍結了。你在第 N 次對話告訴它「我喜歡用咖哩粉而不是咖哩塊」，它在 N+1 次對話完全沒印象。它擁有 175B 參數的「過去記憶」（pretraining 時看過的世界），但沒有形成新長期記憶的能力。

短期記憶呢？有，那叫 context window。但 context window 是個房間，不是腦——你關門離開，房間就空了。

§1.2 Decade of Agents——四大缺口

Karpathy 把當前 agent 的問題列為「2026 年是 agent 元年」這種樂觀派的反駁。他說這應該叫 **Decade of Agents**——要十年才能讓 agent 達到「員工 / 實習生水準的可靠性」。

四大缺口（前三項是他在 Dwarkesh Patel podcast 訪談中明確列出，記憶在 podcast 與 X 推文中均被多次提及作為跨缺口的核心問題；以下整合為四項）：

缺口	現狀
Multimodality	圖像、語音、影片整合仍邊緣
Computer use	瀏覽器、檔案、IDE 操作不可靠
Continual learning（記憶）	訓練後無法 consolidate 新知識
Cognitive capacity	"They're cognitively lacking and it's just not working."

> "It will take about a decade to work through all of those issues."

四項裡最尖銳的是記憶。其他三項是能力延伸，記憶是架構性缺陷——沒有持續性，所有「員工級」的承諾都不成立。一個記不住昨天交辦事項的實習生，沒有公司會要。

§1.3 四種解法的覆蓋範圍

業界當前的解法在 cover 不同的時間跨度——四種主要路線：

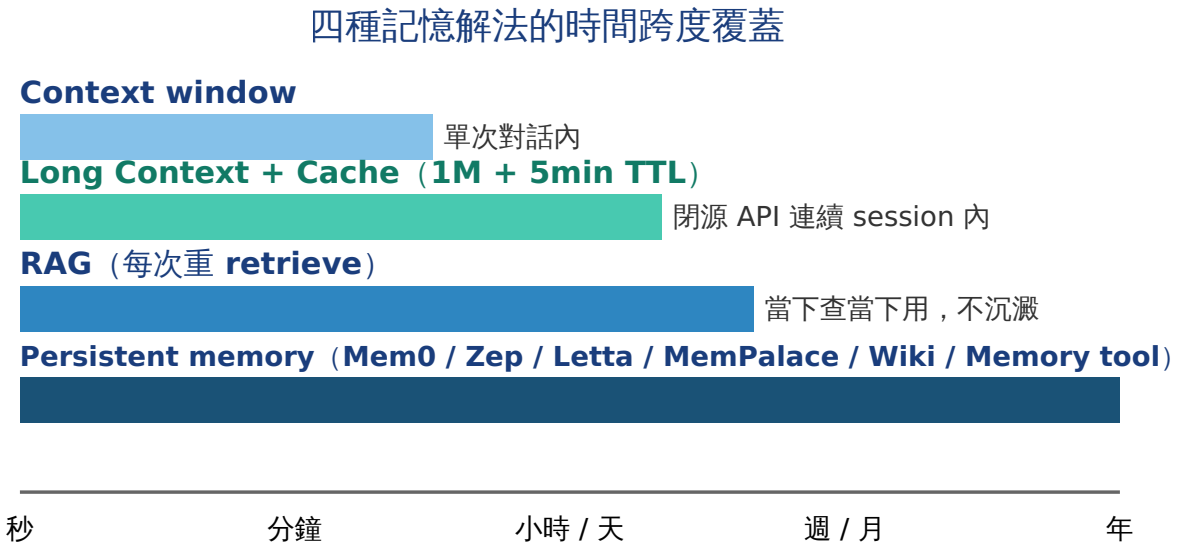


圖 1：四種記憶解法的時間跨度覆蓋——context window 撐單次對話；Long Context + Cache 把「房間」放大到 1M tokens 並透過 cache 跨數分鐘 session 復用，但僅閉源 API；RAG 臨時拉資料不累積；persistent memory 才真正跨 session、跨月、跨年。

Context window 即使做到 1M tokens，仍然是「房間」不是「腦」——關門就空。但 Anthropic / Google / OpenAI 推出的 prompt cache（5 分鐘 TTL）讓「房間」可以在連續 session 之間被重用，這是介於 context 與 persistent memory 之間的灰色地帶。對短歷史 chatbot / 同 project coding session（< 500K tokens），這條路比上 memory 框架簡單 10 倍且更便宜（§2.6 詳述）；但僅限閉源 **API（Claude / Gemini / GPT-4.1）**，自部署開源模型 **1M** 的落地可行性差——這是企業 self-host 場景的關鍵限制。

RAG 是個臨時資料庫，每次重新 query 都從零開始 synthesis。只有 **persistent memory** 那條長條真正回答了 **Karpathy** 的問題：跨 session、跨月、跨年的連續性。

但 persistent memory 的實作差異巨大。下一章把這條長條切開來看，並涵蓋三條對照路線。

---

§2 五大流派：橫向對比

> 本節結論
> - 11 個選項分四層抽象：第零層 Long Context+Cache（不上框架）／第一層 tool-primitive（Anthropic / OpenAI Memory）／第二層 service-layer（Mem0 / Zep / Letta / LangMem / MemPalace / Cognee）／第三層 概念範式（Karpathy LLM Wiki）
> - 第二層 5 大主流流派比拚的軸：verbatim ↔ extraction、flat ↔ hierarchical
> - LongMemEval 同口徑：**Zep 63.8% > Mem0 49.0%**；MemPalace 96.6% 是不同 metric，不可直接比
> - MemPalace 4 月爆紅但 benchmark 爭議大，獨立重現只有 88.9% R@10、無 reranking 跌至 60.3%
> - 第三條對照路線（Long Context / Anthropic / OpenAI）會壓縮 service-layer 的市場空間，但不會消滅它

§2.0 速查表

進深度對比前先給一張全景：

框架	創立 / 釋出	開源	核心隱喻	主場景	**GitHub stars**
Mem0	2024 中	✓（Pro 鎖 graph）	Hybrid store（vector + graph + KV）	個人化 chatbot	~48K
Zep / Graphiti	2024 末	✓（Graphiti 開源）	Temporal Knowledge Graph	事實演變的長期 agent	~10K
Letta（前 **MemGPT**）	2023.10 paper / 2024 商業化	✓	OS paging（RAM/Disk）	長 horizon autonomous	~15K
MemPalace	2026-04-08	✓（MIT）	Method of Loci 空間	離線 / 隱私	48hr 22K → 兩週 ~47K
Karpathy LLM Wiki	2026-04-04（gist）	概念	Compiler（編譯一次）	個人 PKM	概念 gist

橫軸把它們排到 verbatim ↔ extraction、flat ↔ hierarchical 兩個維度，分布像本報告 §0 那張小圖：MemPalace 在左下（verbatim + flat-ish）、Letta 在左上（OS-style hierarchical）、Zep 在右上（extraction-heavy graph）、Mem0 在右下偏中（extraction + hybrid）、LLM Wiki 在右上（compiled markdown 是 hierarchical 的一種變體）。

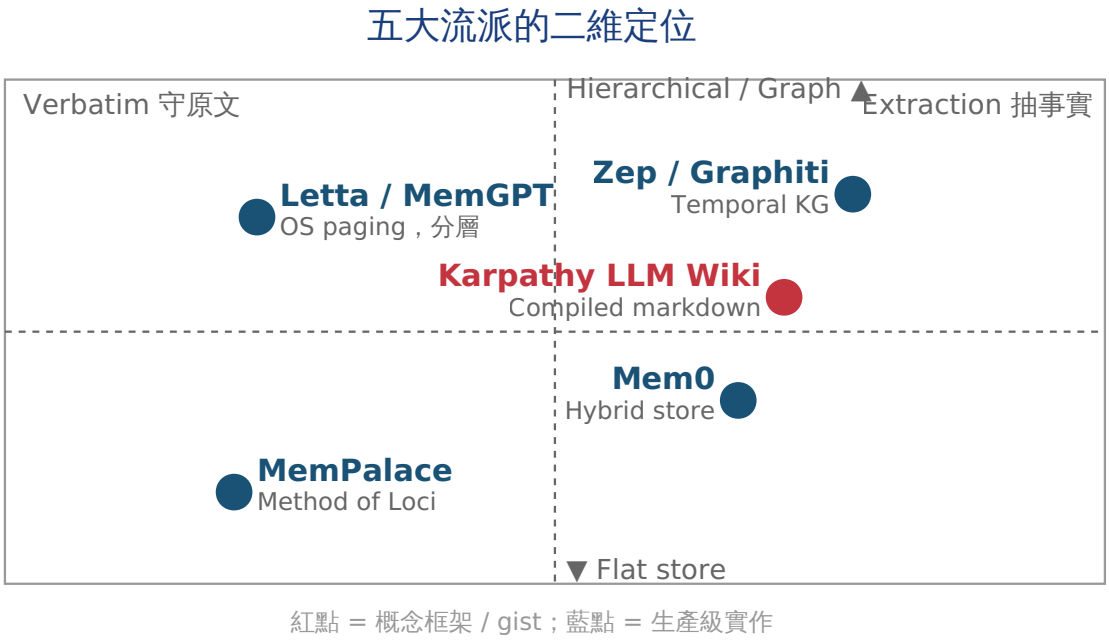


圖 2：五大流派在「儲存哲學 × 結構化程度」二維平面的定位。

§2.1 Mem0 ——社群最大的 hybrid store

Mem0 出生於 2024 年中，是當前社群最大的獨立記憶框架（~48K stars）。它的定位是「drop-in memory layer」——你在原本 stateless 的 agent 外面包一層 service，不需動 agent 邏輯。

架構：

- 三類記憶：episodic（what happened）/ semantic（what is known）/ procedural（how to do）
- **Multi-scope**：`user_id`、`agent_id`、`run_id/session_id`、`app_id/org_id`，retrieval 時 compose
- **Hybrid storage**：vector store（19 個 backend）+ graph variant **Mem0g**（Kuzu / Neo4j）+ KV
- 三種 **hosting**：managed cloud / open-source self-hosted / local MCP

Write path：每次對話結束後 LLM 抽取「這段對話有什麼值得記」→ 寫入 vector / graph。
這是 Mem0 的雙面刃——精準但每次都要付 LLM extraction 成本。

Mem0 自家 LOCOMO benchmark（ECAI 2025，arXiv:2504.19413）：

方法	LLM 分數	P95 Latency	Tokens/conv
Full-context	72.9%	17.12s	~26,000
Mem0g (graph)	68.4%	2.59s	~1,800
Mem0 (vector)	66.9%	1.44s	~1,800
RAG	61.0%	0.70s	—
OpenAI Memory	52.9%	—	—

讀法：Mem0 沒贏 full-context，但在 92% latency 削減 + 93% token 削減的條件下逼近 92% 的準度。這個 **trade-off** 才是 **Mem0** 的真實賣點——「我幾乎跟全塞 context window 一樣準，但便宜 10 倍」。

弱點（Atlan 2026 整理）：
- Graph memory 鎖在 Pro tier

- 無 temporal fact modeling（fact 變了就直接覆蓋，不留歷史）
- **LongMemEval 49.0%（GPT-4o）**——比 Zep 低 15 點

社群最大不等於技術最強。Mem0 贏在生態整合（13 個 agent framework）、Voice agent（ElevenLabs / LiveKit）、SOC 2 Type II 合規，輸在 fact 演變的細膩處理。

§2.2 Zep / Graphiti ——時序知識圖譜

如果你的 agent 需要記住「使用者三個月前說喜歡 cold brew，但上週改說 latte」，Zep 是當前最強的選擇。它的核心隱喻不是 store，是 **temporal knowledge graph**。

Bi-Temporal Model——每個 fact 帶兩種時間戳：

- Event time：這件事何時發生
- Transaction time：這件事何時被記錄 / 失效

> "facts have validity windows. When information changes, old facts are invalidated — not deleted."

舊 fact 不刪除，只標記 invalidated。你可以查「現在使用者偏好什麼」（current state），也可以查「三個月前他說什麼」（historical state）。這正是 Karpathy 在 §5.4 會講到的 episodic memory 的雛形。

儲存模式：
- 三元組（Entity → Relationship → Entity）+ validity window
- 自動從非結構化資料抽取
- 「Everything traces back to episodes — the raw data that produced it」（保 provenance）

Graphiti（開源核心） vs Zep platform（managed）：

層	角色	適合誰
Graphiti	開源 temporal context graph engine	自建偏好者
Zep	managed infra：governance + threads + 預配 retrieval + dashboards	企業 turnkey

性能：query 時不需 LLM call（fact 已抽好），P95 retrieval ~300ms，Atlan 評為「事實演變類場景的最強選擇」。

LongMemEval 63.8%（GPT-4o）——本對照表中最高的一個生產級數字。
口徑警告：是 Zep 自己跑的 GPT-4o 配置，跟 Mem0 那 49.0% 同口徑可比，但跟 MemPalace 那 96.6% R@5 完全是不同 metric（前者測答對率，後者測 retrieval recall）。§4 會專門拆這個。

弱點：社群比 Mem0 小、enterprise 訂價不透明、無 constitutional governance layer（合規仍要自己加）。

§2.3 Letta（前 MemGPT）——agent OS

2023 年 10 月 arXiv:2310.08560 發表的 MemGPT paper 是這場記憶之戰的學術起點。它丟出一個讓人眼睛一亮的隱喻：**LLM 的 context window 像 RAM**，外部儲存像 **disk**，**LLM 自己當作業系統 + page in/out**。

> "inspiration from hierarchical memory systems in traditional operating systems that provide the appearance of large memory resources through data movement between fast and slow memory"

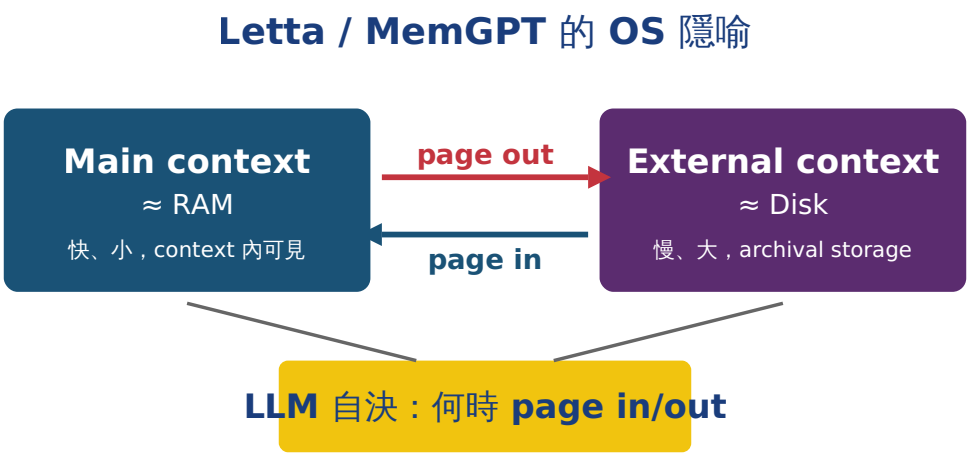


圖 3：Letta / MemGPT 把 LLM 當作業系統，main context 對應 RAM、external context 對應 disk，由 LLM 自己控制 page in / out。

核心差異（vs Mem0 / Zep）：
- Mem0 / Zep 把 agent 當 stateless function，外面包 memory service
- **Letta 把 agent 當 persistent entity**，自己管 state、自己決定何時 archive、何時 retrieve

Memory Block 概念：
- 帶標籤的 memory blocks（如 "human"、"persona"）
- Agent 可在互動中 reference 並 update 各 block
- 支援多 agent 共享 memory blocks（為第二題編排研究預留）

商業化版本（**2024 後 rebrand**）：

產品	對標	場景
Letta Code SDK	Claude Code、Codex	本地 tool execution + skills
Letta API	OpenAI Responses API	純 chatbot
MemFS	（新功能）	git-tracked memory

弱點：非 drop-in，需採完整 agent runtime；社群比 Mem0 小；訂價不透明；無公開 LongMemEval 數字。

為什麼仍重要：對「需要連續跑數天、跨任務累積技能」的 autonomous agent，Letta 是目前唯一原生支援的。Mem0 / Zep 都假設 agent 跑一次就停，Letta 假設 agent 永遠在跑。

§2.4 MemPalace ——4 月份的爆紅與爭議

2026-04-08 釋出，第一波 48 小時內衝到 22K stars，截至 2026-04-29 累積到 ~47K stars。宣稱在 LongMemEval 上達到 96.6% R@5——比 Zep 的 63.8% 高出 33 點。社群一開始震撼，幾天後批評文鋪天蓋地。

核心架構：四層記憶堆疊

Layer	Tokens	載入時機	內容
L0 Identity	~50	啟動時必載	AI persona
L1 Critical Facts	~120（AAAK 格式）	啟動時必載	team / projects / preferences
L2 Room Recall	按需	對話觸發	近期 session 重點
L3 Deep Search	按需	semantic search	全部 compressed memory vaults

170-token 喚醒成本——啟動只需 L0 + L1。對比：raw history 19.5M tokens、LLM-summarized 約 650K。年成本從 ~$507 降到 ~$0.70（純 init）。

空間隱喻（**method of loci**）——這是它命名的根：

- Wing：major categories
- Room：sub-topics
- Hall：shared corridors（facts / events / discoveries / preferences / advice）
- Closet：compressed summaries（AAAK 格式）
- **Drawer**：**verbatim originals**（不丟原文）
- Tunnel：cross-wing connections

雙層儲存哲學：closets 存壓縮 summary，drawers 存原檔。原文可恢復，不像 extraction 派 compress 後就丟。這是它真正的差異化——不是空間隱喻本身，是 verbatim-first。

96.6% 爭議（Nicholas Rhodes 批評文，2026-04-08，11 日更新）：

四點方法論問題：

1. **Hand-tuning**：找出 failing questions 後針對性修補再重測（違反 benchmark integrity）
2. **Retrieval window gaming**：top_k=50 但 dataset 只有 19-32 items → 「memory system 貢獻為零」
3. **Metric confusion**：分數量的是 retrieval presence 而非 answer correctness
4. **API dependency**：100% 配置需要付費 Claude API，但 marketing 強調 local-only

獨立重現：88.9% R@10（hybrid retrieval，無 LLM）；無 reranking 時掉到 60.3%。

> "ChromaDB is doing the heavy lifting, not MemPalace."

批評者也承認真正的創新點：
- Verbatim 儲存哲學
- 階層架構（雖然空間隱喻本身貢獻有限）
- MIT 授權、純 open source、純 local
- MCP 整合 19 個工具

真正的客群——批評者點得很準：「ideal for infrastructure-owning developers, not for operators seeking managed cross-tool memory solutions」。

§2.5 Karpathy LLM Wiki ——編譯器路線

2026-04-04，Karpathy 在 GitHub 上發了一個 1500 字的 gist（karpathy/llm-wiki）。沒有程式碼、沒有 benchmark、沒有 demo——就是一個架構主張。但它在 4 月份引爆的討論量超過 MemPalace、超過 Mem0 的 1.0.4 release，因為它在問一個更根本的問題：當前所有記憶系統的範式對不對？

三層架構：

1. **Raw sources**（不可變）：策展文件——文章、論文、圖、資料檔
2. **The wiki**（**LLM** 維護）：markdown 檔——summaries / entity pages / concept pages / cross-references
3. **The schema**（配置）：一個文件（如 `CLAUDE.md`）定義 wiki 結構、慣例、workflow

核心模式：**incremental**，不是 **retrieval-only**。每個新 source 觸發：

- 閱讀 + 討論
- 寫 summary 頁
- 更新 index
- 修訂相關 entity / concept 頁面（每個 source 約 10-15 頁）
- append log entry

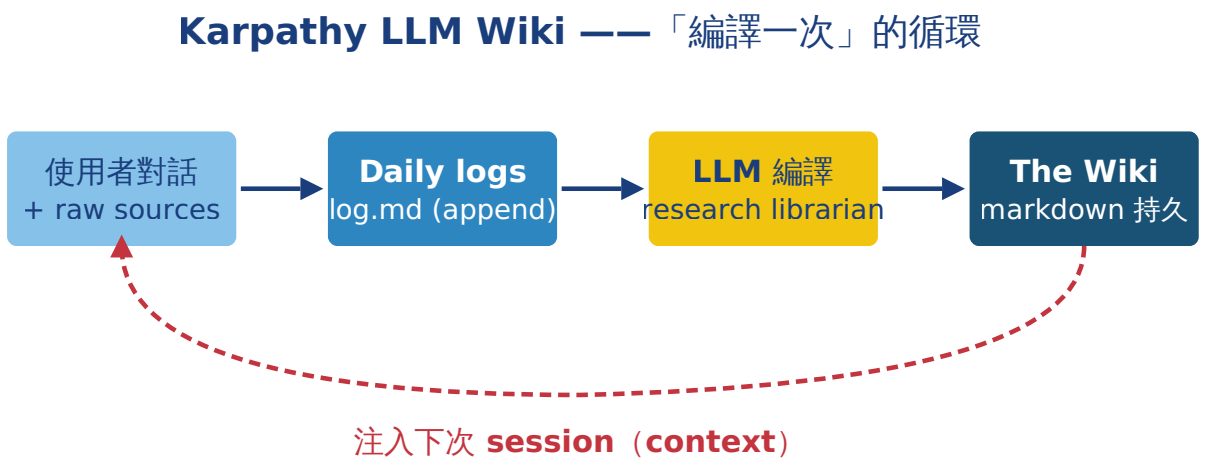


圖 4：Karpathy LLM Wiki 的循環——對話 → daily log → LLM 編譯 → wiki → 注入下次 session。核心差異是「knowledge 編譯一次」而非「每次重新 synthesize」。

最關鍵的對 **RAG** 差異：

> "Most people's experience with LLMs and documents looks like RAG: you upload a collection of files, the LLM retrieves relevant chunks at query time, and generates an answer. This works, but **the LLM is rediscovering knowledge from scratch on every question**."

vs.

> "the LLM **incrementally builds and maintains a persistent wiki**"

LLM 的角色：**research librarian**——不是分析師，是檔案管理員：

> "You're in charge of sourcing, exploration, and asking the right questions. The LLM does all the grunt work — the summarizing, cross-referencing, filing, and bookkeeping that makes a knowledge base actually useful over time."
>
> "Obsidian is the IDE; the LLM is the programmer; the wiki is the codebase."

社群衍生：

- **rohitg00 LLM Wiki v2** ——加 citation validator、rotation policy、conflict resolution
- **Memco / Gamgee / Epsilla** ——商業化嘗試
- **DEV.to「I Over-Engineered Karpathy's Agent Memory」** ——批評實作落地

Karpathy 自承的限制：

- "This document is intentionally abstract. It describes the idea, not a specific implementation."
- index.md 在 ~100 sources / 數百頁規模 work surprisingly well，更大規模需 qmd 或自製搜尋
- 未明確處理 **LLM hallucination / citation grounding**

最重要的本質區別：Mem0 / Zep / Letta / MemPalace 都在做「怎麼把記憶塞進 **query path**」；Karpathy 在做「怎麼讓 **LLM** 自己整理記憶」。前者是 retrieval engineering，後者是 knowledge curation——這是後面 §3.3「Beyond RAG」之爭的根。

§2.6 Long Context + Prompt Cache — context-first 路線

> 這不是 **memory** 框架，是繞過記憶問題的工程路線。但因為它在 **2026** 年實質壓縮了某些 **memory** 框架的市場空間，必須與五大流派並列討論。

核心主張：當你的 model 支援 1M tokens context + cache，你可以把整個對話歷史 / 整個 **codebase** / 整本書直接塞進 context，並透過 prompt cache 在連續 session 之間 5 分鐘內復用——對某些場景，這比上 memory 框架簡單 10 倍且更便宜。

支援 **1M context + cache** 的閉源 **API**（2026-04 現況）：

Provider	模型	Max context	Cache TTL	Cache 折扣
Anthropic	Claude Sonnet 4.5 / Opus 4.7	1M	5 min	寫入 1.25× / 命中 0.1×
Google	Gemini 2.5 Pro / Flash	1M	隱式 + 顯式 cache	命中約 0.25×
OpenAI	GPT-4.1	1M	隱式 cache（自動）	命中 0.5×

為什麼這條路在 **2026** 年突然變成第六派：

1. **5** 分鐘 **cache** 把「房間」變成「準長期記憶」——只要 session 之間隔 < 5 分鐘且 system prompt + 歷史不變，cache hit 後 retrieval cost 接近零、latency 縮 10-50 倍
2. 省掉 **write-time LLM extraction** 成本——不需 Mem0 那種「對話結束後 LLM 抽 fact」的開銷
3. 省掉 **memory framework** 工程複雜度——不需 vector store / graph DB / KG 抽取 pipeline
4. 零資訊損失——verbatim 在 context 中，不像 extraction 派會丟原文

但這條路的邊界很硬，企業落地時必須誠實面對：

限制軸	細節
僅限閉源 **API**	Claude / Gemini / GPT 都是 hosted API；資料離開內網，金融 / 醫療 / 政府 / 國防多數情境不可接受
自部署開源模型 **1M** 落地可行性差	Llama 3.3 多為 128K；Qwen2.5-1M / GLM-4-1M 雖宣稱 1M，但需 **multi-GPU + vLLM/SGLang** 特化部署 + 推論成本是 **128K** 的數倍；且 Needle-in-Haystack 在 > 512K 起出現品質衰退
5 分鐘 **cache TTL**	跨小時 / 跨日的 session 必然 cache miss，full prompt 重算成本是 cache hit 的 10×
無跨 **user** / 跨 **agent** 累積	cache 是單一 session 的快取，不是共享 KB；多 user 場景需要為每人各自維護 context
延遲 / 成本與 **context** 長度線性正相關	1M context 推論成本是 100K 的 10× 起跳；cache miss 的單次調用即可吃掉一個月 Mem0 訂閱
資料敏感性	把整個 codebase / 內部文件 verbatim 送進 hosted API，是合規部門最緊張的場景

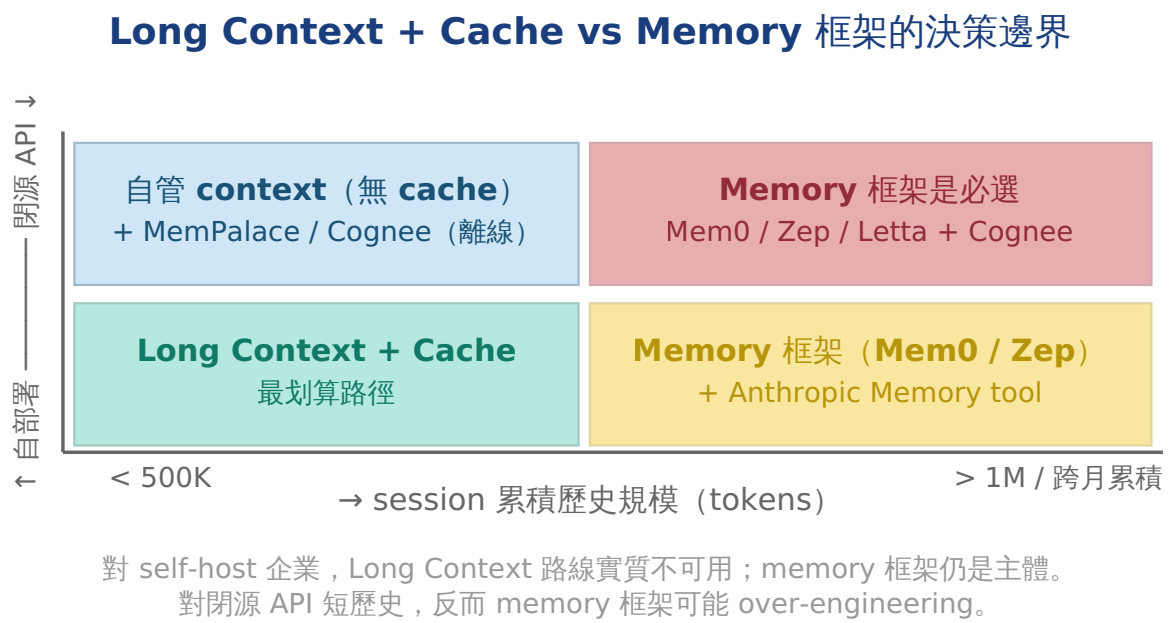


圖 X1：Long Context + Cache vs Memory 框架的四象限決策邊界。「自部署 + 長歷史」與「閉源 API + 長歷史」都仍需 memory 框架；「閉源 API + 短歷史」最划算路徑就是直接吃 Long Context + Cache。

何時用 **Long Context + Cache** 而非 **memory** 框架：

- 短歷史 chatbot（單 session 累積 < 500K tokens）
- 同 project coding session（一個 codebase 全塞 context）
- 一次性深度研究（一本書 / 一份報告整本送進 context）
- 高互動原型（POC 階段不想養 memory pipeline）

何時不能用，必須回到 **memory** 框架：

- self-host / sovereign / regulated industry——資料不能離開內網
- 跨月跨年累積（user 偏好、長期專案知識、agent 學會的技能）
- 多 user / 多 agent 共享記憶池（cache 不能跨 user）
- 推論成本敏感（1M context 不是免費的，每次 cache miss 都是大筆 token 帳單）

Anthropic CTO 視角的真相：Long Context + Cache 不是「memory 的替代品」，而是「memory 的第零層」——對 < 500K 的場景，你不應該先考慮 memory 框架；對 > 1M 或跨月場景，你必須上 memory。這條路把第三方 **memory** 框架的市場壓縮到「**self-host enterprise** + 多 **agent autonomous** + 跨月累積」這三個 **niche**。

§2.7 Anthropic Memory tool + Claude Skills — tool-primitive 路線

如果 Mem0 / Zep / Letta 是「memory 是獨立 service」設計哲學，Anthropic 在 2025 年下半年開始公開的方向是另一條路：**memory** 是 **LLM** 的 **tool primitive**，不是外部服務。

四件套（給 Claude / Claude Code / Claude Agent SDK 開發者的預設組合）：

Primitive	角色	對應第三方框架
Memory tool（`read_memory` / `write_memory`）	LLM 自決何時讀寫；資料以 markdown / JSON 結構化儲存於 Files API	取代 Mem0 的 service layer
Files API	持久檔案系統（含 search、版本）	取代 Mem0 / Zep 的 vector store
CLAUDE.md / 子目錄 CLAUDE.md	專案層級的持久 schema + 規則	取代 LangMem 的 procedural memory
Claude Skills	可組合的能力 primitive，含 memory 操作模式	取代 LangGraph 的 memory orchestration

設計哲學差異：

- **Service-layer** 派（Mem0 / Zep / Letta）：把 LLM 當 stateless function；外面包 memory service；retrieval 在 service 層完成；LLM 只看到拼好的 context
- **Tool-primitive** 派（Anthropic）：LLM 是 stateful agent；memory 是它能呼叫的 tool；何時讀、讀什麼、何時寫，**LLM** 自決；Anthropic 提供 primitive，不提供 retrieval 算法

Karpathy LLM Wiki 在 **Anthropic** 視角下的位置：`CLAUDE.md` + Memory tool 是 LLM Wiki 的工業化版本——schema 用 CLAUDE.md 表達、wiki 內容存於 Files API、編譯流程透過 Skills 規範。Karpathy 的概念框架，Anthropic 的工程落地。

強項：

- 跨 Anthropic 生態一致（同樣 primitive 跨 Claude Code / Claude.ai / API）
- 不需 external service / DB / vector store；對小團隊極友善
- 與 long context + cache 互補（cache 處理短期、Memory tool 處理跨 session）
- 開發者已有的 markdown / git workflow 直接可用

弱點：

- **Provider lock-in**：與 OpenAI / Google 不可移植；多 provider 策略下會卡
- 非通用協議：MCP 雖開源，但 Memory tool 的具體形態仍是 Anthropic 私有 API
- 缺乏跨 **user** 共享 **KB** 原語：仍假設單 user / 單 project
- 無公開 **LongMemEval** 數字：Anthropic 走產品優先路線，benchmark 不是優先

何時優於 **Mem0 / Zep**：

- 你已經 commit 到 Claude 生態（Claude Code / Claude Agent SDK）
- 團隊小、不想養 memory pipeline
- 喜歡 markdown-first / git-trackable 的記憶模式
- 跨 session 但不跨 user 的個人 / 專案場景

何時仍需要 **Mem0 / Zep**：

- 多 LLM provider 策略（不能鎖定 Anthropic）
- 多 user 共享記憶池
- 需要 fact 演變的 temporal model（Anthropic 沒提供）
- enterprise governance（SOC 2 audit trail、合規 dashboard）

§2.8 OpenAI Memory / ChatGPT Personalization — 市佔最大的 memory layer

依使用人數，**ChatGPT Memory** 是當前裝載最多 **end users** 的 **agent memory** 系統——數億 ChatGPT 用戶的偏好、職業、家庭資訊都存在這層。但它在 developer / enterprise 視角的代表性弱，因為它走的是消費者產品而非開發者平台路線。

機制：

- ChatGPT 會在對話中自動抽取「user 個人事實」（職業、城市、興趣、偏好）
- 存於 OpenAI 託管的 user profile 層
- 每次新對話 system prompt 自動注入相關事實
- User 可在 Settings 看到、編輯、刪除每筆 memory
- 跨對話一致，但不跨產品（ChatGPT vs API 是獨立 memory 池）

API 暴露程度：

- ChatGPT 產品：完整 memory 體驗
- OpenAI Platform API：**Responses API** 的 `store: true` + thread state 是接近物
- Assistants API（已 deprecated 進入 Responses）：原本有 thread-level memory
- 沒有獨立的 **`memory.read / memory.write` API**——developer 不能像 Anthropic Memory tool 那樣直接 manage

LongMemEval 表現：52.9%（GPT-4o 配置，第三方測試）——比 Mem0 (49.0%) 高 4 點，比 Zep (63.8%) 低 11 點。不算頂尖，但對「無工程介入、自動運作」的 **baseline** 來說合理。

強項：

- C 端使用者完全透明：use case 不需思考、不需設定
- 與 ChatGPT 產品生態深度整合
- 規模驗證（億級用戶）
- 有用戶介面可看 / 可刪除（GDPR 友善）

弱點 / 不適合場景：

- 黑箱：抽取規則 / 存儲格式 / retrieval 演算法都不公開
- 不可移植：你在 ChatGPT 累積的 memory 帶不走
- 不適合 **self-host enterprise**：本質是託管於 OpenAI 的服務
- 不適合多 **agent** / 多角色：偽單一 user 的 mental model
- **API** 表面太小：開發者拿不到細粒度控制

對開發者的真實啟示：OpenAI Memory 不是給 developer 用的、是給 ChatGPT end-user 用的。它的存在驗證了「memory 是 LLM 產品的標配」這個 hypothesis，但具體的 developer-side 工程選擇仍要回到 Mem0 / Zep / Letta / Anthropic Memory tool / LLM Wiki 這些路線。

§2.9 LangMem — LangChain 生態的 first-class memory

定位：LangMem 是 LangChain 生態（含 LangGraph）為 first-class memory 抽象提供的 module，2024 末釋出，2026 年逐步整合進 LangGraph 主流 API。LangChain 是當前最大的 agent framework 生態，LangMem 是該生態的預設 memory 方案，影響面廣，因此單列一節而非歸入次主流。

三類記憶 **modular API**：

類型	LangMem API	對應
Semantic	`SemanticMemory`	抽 fact 存 vector store
Episodic	`EpisodicMemory`	conversation history with summarization
Procedural	`ProceduralMemory`	self-updating system prompt

強項：

- 生態整合：已在 LangGraph 上的團隊零成本接入
- **Modular**：三類記憶可獨立使用 / 替換
- **Backend** 可換：vector store、graph DB、KV 都支援
- **Procedural memory** 走在前面：自更新 system prompt 是 LangMem 的差異化

弱點：

- **buy-in LangChain** 思想成本高：抽象層多、學習曲線陡
- 生態以外採用率低：不在 LangGraph 上的團隊很少單獨用 LangMem
- 效能 **/ latency** 競爭力不突出：本質仍是 wrapping 各 backend
- 無公開 **LongMemEval** 數字

何時用 **LangMem**：

- 已 commit LangGraph / LangChain
- 多種 backend 並存（要 abstract 起來）
- 重視 procedural memory（self-updating instructions）

何時不用：

- 不在 LangChain 生態，沒理由為 LangMem 引入 LangChain
- 想要 thin wrapper（Mem0 更輕）
- 想要 temporal fact model（Zep 更強）

§2.10 次主流框架（一覽表）

主菜（§2.1-§2.5）+ 三條對照路線（§2.6-§2.8）+ LangMem（§2.9）以外，2026 年活躍的次主流框架：

框架	一句話定位	GitHub	何時考慮
Cognee	Local-first poly-store graph，純離線	~7K	Air-gapped、GDPR 嚴格
Supermemory	MCP-native，整合 Claude Code	—	Coding agent 跨 session
SuperLocalMemory	Local-only 變體	—	個人 PoC
MemoClaw	輕量 wrapper	—	快速原型
ByteRover	LLM-curated hierarchical context	—	學術近實驗

值得注意：**Supermemory** 自報跨 **LongMemEval / LoCoMo / ConvoMem** 全領先，但無第三方驗證——這在 2026 年的 benchmark inflation 浪潮裡屬於「先信一半」的情況。

§2.11 學術原型（ICLR 2026 MemAgents Workshop）

ICLR 2026 收的 MemAgents workshop 是當前學術前沿匯總：

系統	核心貢獻
HiMem	Hierarchical Long-Term Memory for Long-Horizon Agents
SYNAPSE	Episodic-Semantic Memory via Spreading Activation（人腦聯想啟發）
TiMem	Temporal-Hierarchical Memory Consolidation
MAGMA	Multi-Graph based Agentic Memory Architecture
GAM	Graph-based Agentic Memory（Semantic-Event-Triggered，episodic→semantic consolidation）
A-MEM	Zettelkasten-inspired，dynamic linking
MIRIX	六模組（Core / Episodic / Semantic / Procedural / Resource / Knowledge Vault）
Coarse-to-Fine	多層 memory 支援 exploration + 錯誤修正

emergentmind 整理（公式形式為轉錄自 emergentmind 的整理筆記，並非標準 Ebbinghaus forgetting curve `R = exp(-t/τ)`；雙重指數形式接近 Gompertz survival function，建議讀者直接以核心觀念理解，公式具體形式以原論文為準）：

$$ p(t) \approx 1 - \exp(-r \cdot e^{-a t}) $$

核心觀念：retrieval 機率組合 relevance r、time decay a、頻率三軸。這個模型是當前生產級框架幾乎都沒實作的——大家都靠 vector similarity 直接做，而不是建模 forgetting curve。學術前沿與生產級的 gap 仍很大。

---

§3 設計取捨深度剖析

> 本節結論
> - 三派儲存哲學定位：**Verbatim** 守原文（MemPalace）／**Extraction** 抽事實（Mem0 / Zep）／**Compiled-Wiki** 編譯結構（Karpathy）
> - Write-time vs Read-time 成本是所有框架的第一個分岔——Mem0 / Zep 寫貴讀快，MemPalace 寫零成本但讀靠 retrieval engine
> - RAG vs Wiki 不是替代關係：RAG 適合 large dynamic corpus，Wiki 適合 personal / project knowledge
> - 中文場景所有框架都是英文中心，需自行投資 **NLP** 中介層（embedding 換 BGE-zh、entity normalization、temporal layer）

§3.1 Write-time vs Read-time 成本

這是所有記憶框架的第一個分岔。Write-time 成本指寫入記憶時的代價（通常是 LLM call、CPU、儲存）；read-time 成本指查詢時的代價（latency、token）。

框架	Write 成本	Write 需 LLM	Read latency	Read 需 LLM
Mem0	高	✓（每次 extraction）	1.44s	✗
Zep / Graphiti	中	✓（KG 抽取）	~300ms	✗
Letta	中	✓（agent 自決 archive）	變動	✓（page in 時）
MemPalace	零	✗（verbatim 直存）	變動（L0/L1 即時）	✗
LLM Wiki	中	✓（research librarian 編譯）	變動（markdown 直查）	部分

兩個極端：
- **Mem0 / Zep**：寫入很貴（LLM 抽事實 / 三元組），但寫完之後 read 飛快、不需 LLM
- **MemPalace**：完全不付 write LLM 費用，但 read 時要靠 retrieval engine（ChromaDB）+ 階層走訪

LLM Wiki 是個第三條路：write 時付 LLM 編譯費，但這是攤提的——一次編譯，N 次 read 受惠。對 personal 場景特別划算（一個人一天對話量小，編譯成本可接受）。

§3.2 Verbatim vs Extraction vs Compiled-Wiki ——三派哲學

如果把「資訊進入記憶系統時被怎麼處理」當作維度，2026 年清楚分成三派。

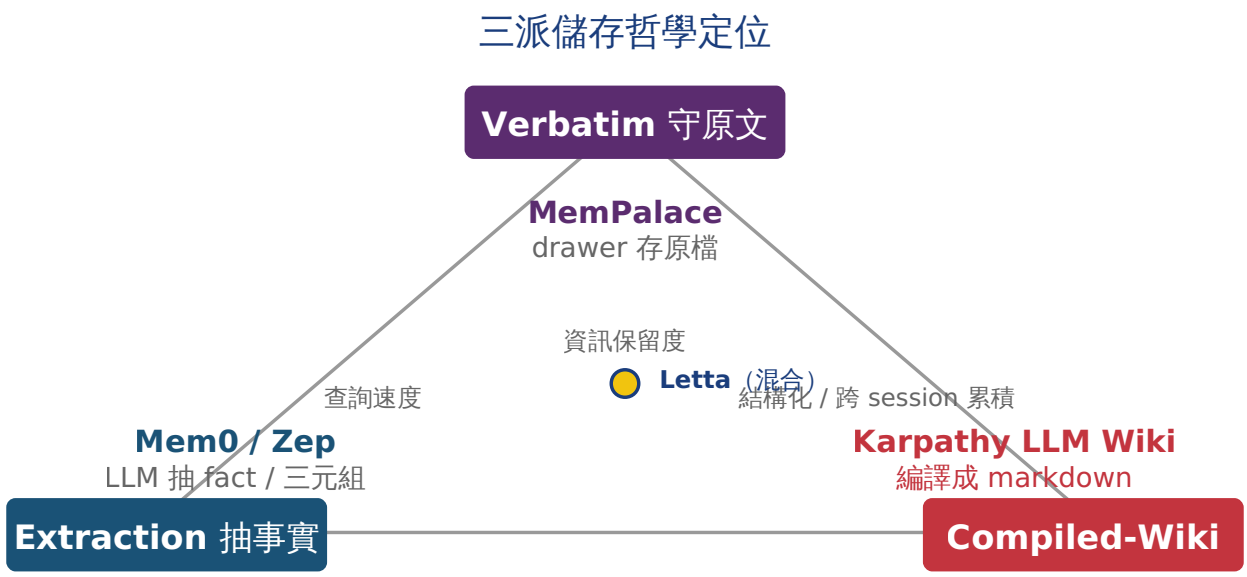


圖 5：三派哲學的三角定位——Verbatim（守原文）、Extraction（抽事實）、Compiled-Wiki（編譯成結構化文件）。Letta 因為 main context 是 verbatim、external 是 extraction，落在三角中間。

三派的本質差異：

維度	Verbatim	Extraction	Compiled-Wiki
寫入時資訊損失	無（原文保存）	高（fact 抽完丟原文）	中（編譯到 wiki，但 raw sources 可保留）
查詢速度	中	快	中（markdown 直查）
跨 source 結構化	弱	中（KG）	強（cross-references）
對 LLM hallucination 的脆弱度	低	高	中
寫入成本	零 **LLM**	高	中

Verbatim 派的賭注：原文不丟，retrieval 找對片段就夠了，不需要 LLM 預先理解。像紙本檔案櫃。
Extraction 派的賭注：LLM 抽出來的 fact 就是「真實有用的資訊」，原文是雜訊。
Compiled-Wiki 派的賭注：真正有價值的不是 fact 或原文，是人 **+ LLM** 一起整理出的結構化知識——這層結構是 wiki 的本體。

三派的選擇本質上是對「知識本質」的不同假設。Karpathy 寫 wiki gist 時，本質上在反對 extraction 派——他認為單純抽 fact 是不夠的，因為 fact 之間的關聯、上下文、層次，才是真正讓知識可用的東西。

§3.3 Beyond RAG ——Karpathy 為何說 RAG 不夠

「RAG is dead, long live the Wiki」——4 月以來這句話在 X / Hacker News 反覆被引用，源頭是 Karpathy 的 gist。但 RAG 真的死了嗎？

Karpathy 的論點：

> "the LLM is rediscovering knowledge from scratch on every question"

每次 query 都從 raw chunks 重新 synthesize 答案。這意味著：
1. 沒有複利——你問過 100 次的東西，第 101 次仍從零開始

2. 沒有跨 **query** 一致性——同樣問題不同時間問可能拿到不同答案
3. 沒有結構化沉澱——零散 chunk 無法組成完整圖譜

Wiki 路線的解法：knowledge 編譯一次，cross-references 已就位，每次 query 直接讀已 synthesize 好的 markdown。

反對者的論點（社群中的 RAG 派）：

1. **Wiki** 維護成本不可控——LLM 自維護的可靠性無 benchmark，hallucination 會被「永久化」
2. **Wiki** 無法處理快速變動的資料——RAG 對 raw sources 直查，wiki 一旦過時就誤導
3. **Wiki** 的 **schema** 是負擔——你要先想好怎麼組織，這個前置成本對非典型場景很高
4. **DEV.to** 的批評文證明：在 client-side 環境（Claude Code、本地筆電）實作 wiki 的 background process 失敗率 ~50%

真實情況：兩條路解的不是同一個問題。

- **RAG** 適合：query 對 large dynamic corpus（文件多、更新頻、查詢面廣）
- **Wiki** 適合：personal / project knowledge（querie 集中、累積性強、需跨時間一致）

對個人工作流（我研究的這個主題、我寫的這個專案、我的個人偏好），Wiki 大幅優於 RAG。
對企業搜尋、客服 KB、即時資料查詢，RAG 仍是預設解。

Karpathy 的真實主張其實是：「對個人 **/ agent** 持續學習的場景，當前社群押錯方向了，預設應該是 wiki 而非 RAG」——而不是「RAG 全面該死」。媒體標題化讓這個 nuance 流失了。

§3.4 中文 / 跨語言場景的限制

所有主流框架都是英文中心。中文場景的真實落地門檻比想像的高，分四層問題拆解：

問題 **1**：**Embedding** 模型品質

預設 **embedding**	中文 **retrieval recall**（業界估計）	中文友善替代	替代後 **recall** 改善
`all-MiniLM-L6-v2`（MemPalace 預設）	比英文低 15-25 點	BGE-large-zh-v1.5	+12-18 點
OpenAI text-embedding-3-large（Mem0 / Zep 常見）	比英文低 5-10 點	jina-embeddings-v3-zh	+5-8 點
Voyage-3（Anthropic 推薦）	比英文低 8-12 點	BGE-M3（多語）	+4-7 點

注意：以上數字是業界常見估計，本研究未實測。建議任何中文 agent 上線前用自家 query 集做 ablation。

問題 **2**：**Entity extraction** 失誤率

中文對話抽 entity / fact 比英文難的具體場景：

- 人名地名混淆：「王小明」是人名、「中山路」是地名、「張中山」可能兩者皆是；當代中文 NER 模型在閒聊體 transcript 上 F1 約 0.75-0.85（英文 0.92-0.95）
- 代詞 / 省略：「他昨天說的那個事」涉及指代消解，LLM extraction 容易把 entity 拼錯
- 稱謂變化：「小王 / 王老師 / 王經理 / 老王」可能指同一人，需 entity linking

問題 **3**：時序表達不規範

KG-based 框架（Zep / Graphiti）對 temporal extraction 很敏感，但中文時序表達高度上下文依賴：

中文表達	標準化困難	建議處理
「上週」「前年」「兩個月前」	需綁定當下日期	LLM 預處理 → ISO 8601
「春節前」「中秋連假」	文化曆 + 年份依賴	維護中文節慶映射表
「我那時還在念大學」	模糊區間	視 entity 額外屬性處理
「最近」「之前」	高度主觀	標註 fuzziness，retrieval 時放寬 window

問題 **4**：**Tokenizer** 浪費 **+** 成本爆

- GPT / Claude / Gemini 的 BPE tokenizer 對中文不友善：1 個漢字常吃 2-4 tokens
- 同樣語意的中文 prompt 比英文貴 1.5-3×
- 長 context 場景（§2.6）成本放大效應更大

台灣 **/** 中文團隊的工程建議：

1. 預設換 **embedding**：BGE-zh / jina-zh 用一次就回不去 OpenAI 預設
2. 加 **LLM** 中介層做 **entity normalization**：對話進 memory 前先過一輪 LLM 整理（人名統一、時序正規化）
3. 慎選 **temporal-aware** 框架：Zep / Graphiti 對中文 fact 演變的健壯度仍待實測；對時間敏感的應用（醫療紀錄、法律案件）建議自建 temporal layer
4. **Cost monitoring** 比英文場景更重要：中文 token 成本被低估時很快會超預算
5. 本地化 **benchmark**：LongMemEval / LoCoMo 都是英文 dataset，沒有中文版可信第三方 benchmark；台灣 / 中國團隊應自建 query 集做 internal eval

結論：沒有任何主流框架預設處理好中文場景。對台灣 **/** 中文市場開發 **agent** 的團隊，這層是必須自己投資的工程，且工作量比想像的大。能力允許的話，把中文 NLP layer 視為 memory 框架之外的獨立 capability 來投資，不要假設換 embedding 就解決。

---

§4 Benchmark 解讀：別被數字騙了

> 本節結論
> - LongMemEval 是當前最佳對照基準，僅代表「閒聊體 **long-context** 場景」，對 coding agent / domain-specific / 多 agent / 中文場景代表性低
> - 跨家數字直接比是不嚴謹的——每家用不同 model + embedding + dataset，**MemPalace 96.6%** 是 **retrieval recall**，**Mem0 49.0%** 是 **answer correctness**，本質不可比
> - 2026 年 benchmark inflation 嚴重，每家都在自家口徑下宣稱最強
> - 採購原則：(a) 第三方獨立驗證 (b) 口徑一致 (c) 在你的場景重現
> - 在自家真實 **task** 上跑 **ablation** 永遠優於信第三方 **benchmark**

§4.1 LongMemEval / LoCoMo 對照

框架	Benchmark	Model	Score	來源
Mem0 (vector)	LOCOMO	GPT-4o	66.9% LLM Score	Mem0 自家 paper
Mem0g (graph)	LOCOMO	GPT-4o	68.4%	Mem0 自家 paper
Mem0	LongMemEval	GPT-4o	**49.0%**	第三方（Atlan）
Zep	LongMemEval	GPT-4o	**63.8%**	Zep 公開
MemPalace	LongMemEval	mixed	96.6% R@5（爭議）/ 88.9% R@10（獨立重現）/ 60.3%（無 reranking）	MemPalace 自家
Letta	—	—	無公開	—
Cognee	—	—	無公開	—
LangMem	—	—	無公開	—
Supermemory	LongMemEval / LoCoMo / ConvoMem	—	自報領先	無第三方

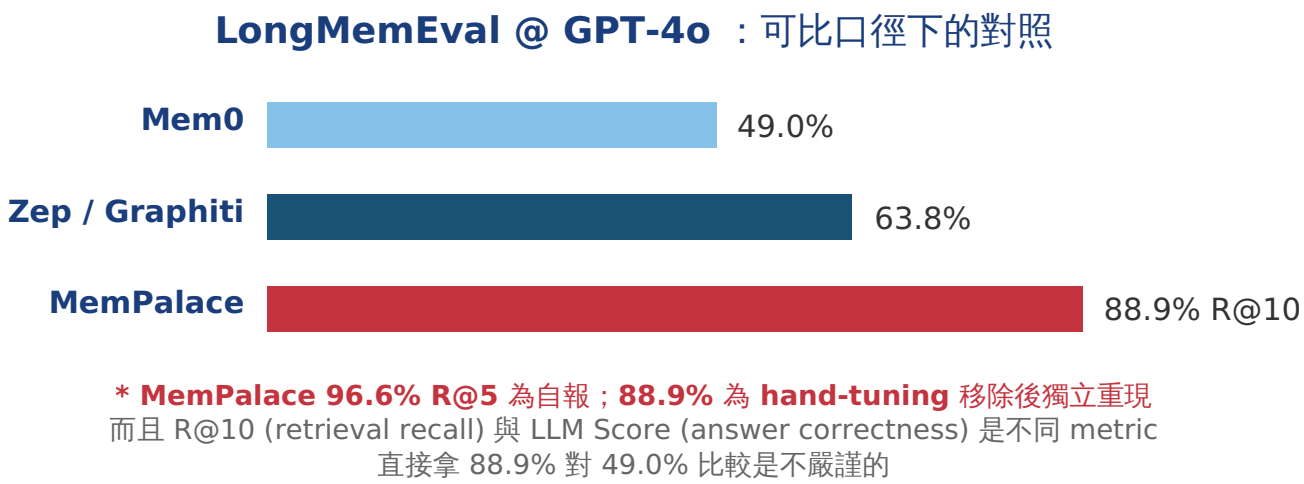


圖 6：LongMemEval 對照——Mem0 49% vs Zep 63.8% 是同口徑可比；MemPalace 88.9% 是不同 metric，須加註腳。

§4.2 為何 96.6% 不可直接拿來打 49.0%

四個口徑差異：

1. **Metric 不同**：MemPalace 量的是 **R@5（retrieval recall@top5）**——找到正確 chunk 的機率；Mem0 / Zep 量的是 **LLM Score（answer correctness）**——拿到 chunk 後最終答對的機率。一個是「找到」、一個是「答對」，找到不等於答對。
2. **Top-K gaming**：top_k=50 但 dataset 只有 19-32 items，等於「全部都拿出來」，retrieval 系統幾乎沒在過濾。
3. **Hand-tuning**：MemPalace 團隊在初版發布前已對 failing questions 做針對性修補。
4. **Model 不同**：Mem0 / Zep 都用 GPT-4o，MemPalace 用 mixed config。

真實可比的 **number**：MemPalace 的 60.3%（純 retrieval、no reranking、no LLM）vs Mem0 的 49.0%。仍領先約 11 點，但不再是 47 點的天文差距。

更深的教訓：2026 年 agent memory 的 benchmark 戰場已經 inflation。每家都在自家口徑下說自己最強。買產品 / 選框架時，別看 **marketing** 數字，看 **(a)** 是否第三方獨立驗證、**(b)** 口徑是否一致、**(c)** 是否在你的場景重現。

§4.3 LongMemEval / LoCoMo 自身的 dataset bias

跨家口徑差異拆完了，還有更根本的問題：這些 **benchmark** 本身的 **dataset** 偏在哪裡？

LongMemEval（Wu et al., 2024）：

- 由 ~500 個 long-conversation scenarios 組成
- 任務型態：multi-turn dialogue，使用者 / 助手對話累積大量背景，最後問 5 類 question（single-session-user / single-session-assistant / multi-session / temporal-reasoning / knowledge-update）
- **Dataset bias**：以「閒聊體 + 個人資訊累積」為主，類似 ChatGPT Memory 的典型情境

LoCoMo（Maharana et al., 2024）：

- 模擬兩個 persona 之間長達數十 session 的對話
- 重點在「跨 session fact recall」與「causal reasoning」
- **Dataset bias**：仍是對話 / persona 為主

這對 **benchmark** 結果代表什麼：

你的真實場景	這些 **benchmark** 代表性	補強建議
個人化 chatbot / 客服	高	直接信 benchmark 排序
長 horizon autonomous agent	中	用 benchmark + 自家 task simulation
AI Coding Agent（跨 session 記住 codebase 結構）	低	自建 coding-task memory benchmark；現有數字參考度差
法律 / 醫療 / 金融 domain agent	低	自建 domain-specific benchmark；現有數字幾乎不適用
多 agent 協作 / shared memory	無	完全沒對應 benchmark
中文場景	無	自建中文 query 集

結論：LongMemEval / LoCoMo 是「閒聊體 long-context 場景」的代表性 benchmark，不是 agent memory 的通解 benchmark。對 coding / domain-specific / 多 agent / 中文場景，現有 **benchmark** 排序基本不能拿來決定選型。Anthropic CTO 視角的建議：在自家真實 task 上跑 ablation 永遠優於信第三方 benchmark。

---

§5 認知架構視角

> 本節結論
> - 人腦記憶四元論（**Working / Episodic / Semantic / Procedural**）是當前所有框架的設計原型
> - 5 大主流框架在四類記憶的支援度差異大：MemPalace 弱在 procedural，LLM Wiki 弱在 episodic 自動化
> - Karpathy 的核心主張：當前缺一個「**System Prompt Learning**」範式——LLM 會自己寫筆記、總結教訓
> - Spreading activation（人腦聯想擴散）是 retrieval 下一條路，但生產級框架尚未原生支援

> - 三階段 consolidation（**Episodic → Semantic → In-Weights**）才是 Karpathy 真實野心，最終要把記憶烤回權重——若這成立，現有所有框架都是過渡形態

§5.1 Working / Episodic / Semantic / Procedural ——四元論

人腦記憶的心理學分類，當前 agent memory 的設計幾乎都在參考：

類型	人腦	**Agent** 對應	框架支援度
Working	短期、即時、容量小	Context window	全部都支援（即 LLM 本身）
Episodic	過去事件的回憶	對話歷史 / interaction log	Mem0 ✓ / Zep ✓ / Letta ✓ / MemPalace ✓ / Wiki 部分
Semantic	抽象事實、概念	結構化 fact / KG / 概念頁	Mem0 ✓ / Zep ✓ / Letta 部分 / MemPalace ⚠ / Wiki ✓
Procedural	怎麼做（技能、習慣）	system instruction / 工具調用 pattern	LangMem ✓ / Mem0 v1.0.0+ ✓ / Letta ✓ / 其他多缺

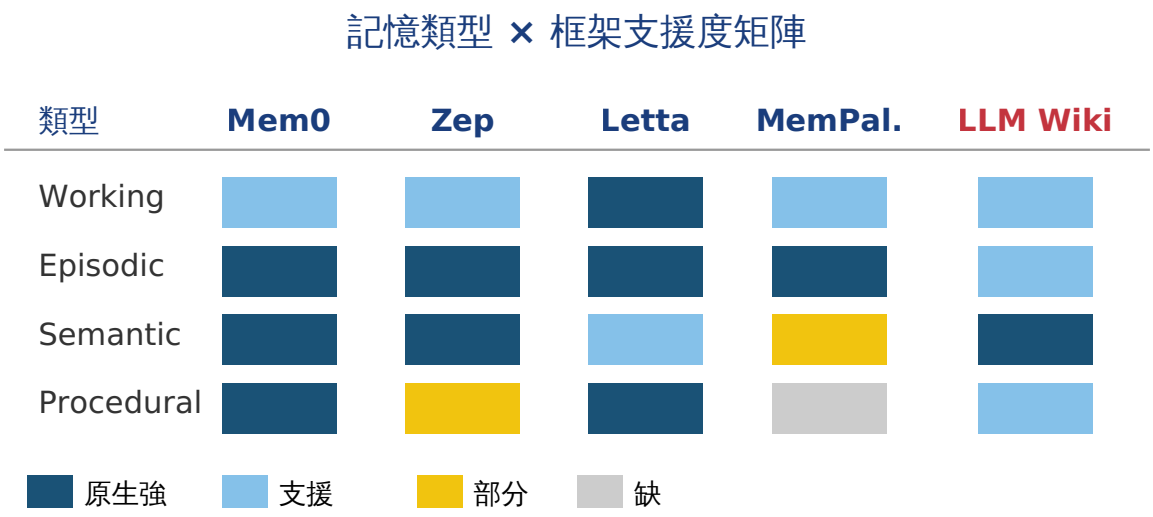


圖 7：記憶類型 × 框架支援度——四類記憶在五大框架的成熟度差異。MemPalace 弱在 procedural（無自更新行為）；LLM Wiki 弱在 episodic 自動化（要 daily log 編譯流程）。

詳細實作方式：

類型	Mem0	Zep	Letta	MemPalace	LLM Wiki
Working	LLM 內部	LLM 內部	Main context（OS RAM）	L0+L1（170 tokens）	注入的 wiki 段
Episodic	session 記錄	KG episodes	Conversation buffer	Hall (events)	Daily log
Semantic	extracted facts	KG nodes + validity	Memory blocks	Closet (compressed)	Wiki entity pages
Procedural	v1.0.0+ procedural	（弱）	Block update	（無）	schema (CLAUDE.md)

§5.2 System Prompt Learning ——Karpathy 提的第三範式

2025 年 5 月 Karpathy 在 X 丟下這段話：

> "We're missing (at least one) major paradigm for LLM learning. Not sure what to call it, possibly it has a name - system prompt learning? Pretraining is for knowledge. Finetuning (SL/RL) is for habitual behavior. Both of these involve a change in parameters but a lot of human..."

他在指當前 LLM 缺一個學習範式：

範式	機制	對應人類
Pretraining	改參數	從小到大累積知識（小學到大學）
Finetuning（SL/RL）	改參數	養成習慣 / 個性（職場磨練）
System Prompt Learning	不改參數，改顯式規則 / 筆記 / 反思	寫筆記、總結教訓、列規則（成人學習）

人類學新東西的主要方式不是「重訓大腦」，而是寫筆記、列清單、跟自己對話。當前 LLM 沒有這個機制——所有「我學到一個新教訓 → 把它寫成下次要遵守的規則」的路徑，都需要人類介入（手動修 prompt、手動 fine-tune）。

LLM Wiki 是 System Prompt Learning 的具體落地——agent 自己寫筆記、自己編譯結構化文件、下次 session 自己讀。Karpathy 把抽象的範式主張變成可執行的架構原型。

這也解釋了為什麼 2026 年所有 memory 框架感覺像「很複雜的 RAG」——它們都還沒跨過這條範式線。真正的下一代不是「更好的 **retrieve**」，是 **agent** 學會自己寫 **procedure**。Mem0 v1.0.0 加 procedural memory、LangMem 推自更新 system instruction，都是在這條線上的試探。

§5.3 Spreading Activation ——SYNAPSE 路線

人腦 retrieval 不是按 vector similarity，是按聯想擴散——你想到「咖啡」，腦中會自動激活「早晨」「義式」「下雨天」「困倦」等相關概念，激活強度隨關聯距離衰減。

ICLR 2026 的 SYNAPSE paper 把這個機制建模到 agent memory：

- 每個 memory node 有 activation level
- Query 觸發初始 activation
- Activation 沿 graph edges spread，按距離衰減
- 多個 query 的 activation 累加
- Top-k highest activation 作為 retrieval 結果

跟 vector similarity 的差異：
- Vector similarity：query 和 node 的「直接距離」
- Spreading activation：query 透過 graph 結構間接激活的所有 node

實際效果：對「上週我們討論的那個專案，後來怎麼樣了」這種多跳關聯的 query，spreading activation 顯著優於 vector similarity。但成本是 graph 結構必須夠豐富、edges 要夠準。這也是為什麼 **Zep** 走 **KG** 路線在這類場景優於 **Mem0** 走 **vector** 路線。

當前生產級框架幾乎都還沒原生支援 spreading activation。Zep 的 graph traversal 算近似版，但 activation level 的累加 / 衰減模型不完整。學術前沿與生產級的 12-18 個月 gap，這是其中一例。

§5.4 Episodic → Semantic → In-Weights ——三階段 Consolidation

人腦睡眠的核心功能之一是記憶 consolidation：白天的 episodic memory（具體事件）在睡眠中被 hippocampus 重播，逐步轉成 cortex 的 semantic memory（抽象規則 / 概念）。多年下來，最常用的 semantic memory 進一步「程序化」，變成不需思考的 implicit knowledge。

當前 agent memory 系統大多只 cover 第一段（episodic），少數 cover 到第二段（semantic）。**Karpathy** 的真實野心是直通第三段——把 wiki 烤回模型權重。

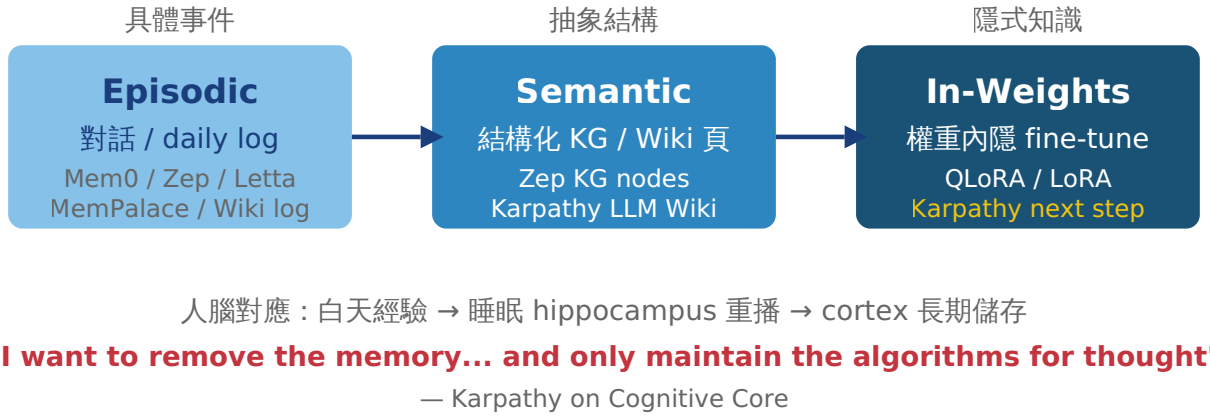


圖 8：三階段 consolidation——對應人腦的 episodic → semantic → in-weights 路徑。當前 agent memory 主要在第一、二階段，Karpathy 預告 next step 是把 wiki 烤回權重（第三階段）。

為什麼第三階段重要：

- 第一階段（**episodic**）：每次都查 log，慢、貴、易雜訊
- 第二階段（**semantic**）：查結構化文件，較快但仍需 context window 載入
- 第三階段（**in-weights**）：知識直接內嵌權重，零 **retrieval cost**、零 **latency**，跟 pretraining 同等地位

但第三階段有兩個未解問題：

1. **Continual fine-tune** 的 **catastrophic forgetting**——學新東西會壞掉舊能力
2. 個人 **vs** 通用的權衡——把使用者個人記憶烤進權重，等於每個人需要自己的 personal model

Karpathy 給的方向：Cognitive Core——瘦核心（~1B 參數）+ 外掛記憶 + 個人化 fine-tune。

> "I want to remove the memory... and only maintain the algorithms for thought"

如果這個願景成立，當前的 Mem0 / Zep / Letta 全部是過渡形態。它們在解第一、二階段，沒解第三階段。最終的 **agent memory** 不是更好的外掛記憶服務，而是讓記憶部分變回權重的一環。

§9.9 會深入這個推論。

---

§6 與 MCP / A2A 編排層的邊界

> 本節結論
> - Agent 系統三層堆疊：**Capability**（LLM / Cursor / Devin）→ **Orchestration**（MCP / A2A / LangGraph）→ **Memory**（Mem0 / Zep / Letta / MemPalace / Wiki）
> - 三層理論獨立、實務常重疊：Letta 越界進編排層、MCP 越界進記憶層、Karpathy LLM Wiki 同時跨三層
> - 多 agent 共享 memory 是當前最弱的環節——所有 service-layer 框架都假設單 agent / 單 user
> - 第二題（多 agent 編排研究）的核心問題：memory 該不該離開 agent 框架成為獨立 service

這份報告聚焦在「記憶層」，但 agent 系統還有「編排層」（orchestration）——多個 agent 怎麼協作、怎麼共享 context、怎麼分工。MCP（Model Context Protocol）和 A2A（Agent-to-Agent）是當前最受矚目的兩個編排協議。

三層堆疊：

層	代表	負責	不負責
Agent Capability	Claude Code、Cursor、Devin、各家 LLM	推理、tool use、執行	跨 session 持久 / 跨 agent 協作
Orchestration	MCP、A2A、LangGraph、AutoGen	多 agent 協作、shared context、tool 註冊	長期記憶、跨 session 知識累積
Memory	Mem0、Zep、Letta、MemPalace、LLM Wiki	跨 session 持久、知識累積、個人化	即時 tool 調度、agent 間訊息路由

三層理論上獨立，實務上有重疊：

- **Letta 越界進編排層**：原生支援多 agent 共享 memory blocks
- **MCP 越界進記憶層**：Anthropic Memory tool 是 MCP 上的 memory primitive
- **Karpathy LLM Wiki 同時在三層**：`schema.md` 是 capability 配置 + wiki 是 memory + agent 互讀 wiki 是編排

對選型的意義：

你的需求	落在哪層	對應產品
單 agent、跨 session 記住使用者偏好	Memory only	Mem0 / MemPalace
多 agent 共享 context	Memory + Orchestration	Letta + MCP
跨團隊 / 跨組織 agent 通訊	Orchestration only	A2A
個人 PKM、跨工具一致	Memory + 部分 Capability（schema）	LLM Wiki

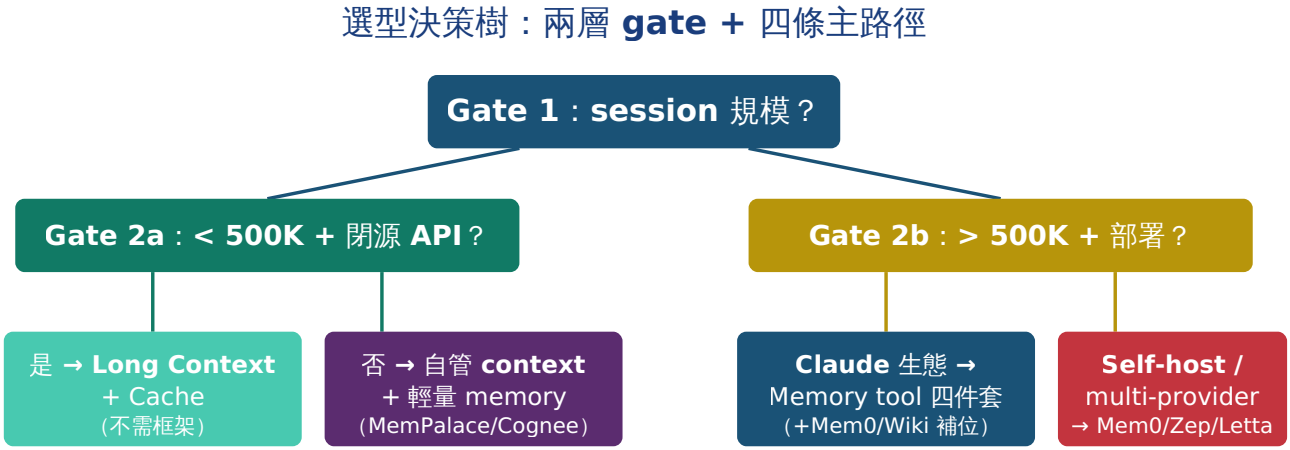
為第二題（多 **Agent** 編排）預留：當你的需求是「多個 agent 各自跑、需要共享同一個記憶池」，問題就從「memory 框架選型」升級到「memory 該不該離開 agent 框架成為獨立 service」。Letta 的 memory block 是嘗試一，MCP-based shared memory 是嘗試二，但都還在早期。第二題會深入這個邊界。

---

§7 選型決策樹

> 本節結論
> - 兩層 gate 決策：**(1) session 規模 → 是否需要 framework**；**(2) 部署環境 → Claude 生態 or multi-provider**
> - 四條主路徑：(a) Long Context + Cache、(b) 自管 context + 輕量 memory、(c) Anthropic Memory tool 四件套、(d) Mem0 / Zep / Letta self-host
> - 第三方 memory 框架（Mem0 / Zep / Letta）的真實 niche：**self-host enterprise + 跨月跨年 + 多 agent autonomous**
> - 選 Mem0 / Zep 之前先問：Long Context、Anthropic Memory tool、OpenAI Memory 這三條路是否都不適用
> - 細節分流（事實演變→Zep / 個人化→Mem0 / Autonomous→Letta / 離線→Cognee/MemPalace）屬第二級決策

本章把選型拆成兩層 gate + 四條主路徑。第一層 gate 決定要不要上 memory 框架，第二層 gate 決定走 Claude 生態還是 multi-provider，最後才到 Mem0 / Zep / Letta 的具體場景分流。



「事實演變→Zep」「個人化→Mem0」「Autonomous→Letta」「離線→Cognee/MemPalace」
屬 Self-host/multi-provider 路徑下的細部分流，見場景對應速查表。

對 self-host 企業，Long Context 路線實質不可用，memory 框架仍是主體。

圖 9：兩層 gate 決策樹——先看 session 規模決定要不要上 memory 框架，再看部署環境決定走 Claude 生態還是 multi-provider；最後才到 Mem0 / Zep / Letta 的具體場景分流。

場景對應速查：

場景	第一推薦	第二選擇	何時不需要 **memory** 框架
短歷史 chatbot（< 500K tokens）	**Long Context + Cache**（Claude / Gemini）	Mem0 light	session 累積 < 500K + 閉源 API 可用
同 project coding session	**Long Context + Cache**（codebase 全塞）	Anthropic Memory tool（CLAUDE.md）	單 project / 短 sprint
Claude 生態跨 session 開發者	**Anthropic Memory tool** 四件套	+ Mem0（補多 user）	預設選項；不需 external service
個人化 chatbot / B2B copilot（multi-provider）	**Mem0**	LangMem	—
客服 KB（事實隨時間變）	**Zep / Graphiti**	Mem0g	—
跨任務累積技能的 autonomous agent	**Letta**	—	—
法律 / 金融知識追蹤	**Zep**	Cognee	—
Self-host / sovereign 企業	**Mem0 + Cognee** 或 **Letta self-hosted**	MemPalace（純離線）	必選 **memory** 框架（Long Context 不可用）
Air-gapped 企業	**Cognee** + Ollama	MemPalace	大廠 cloud 解不可用
個人 PKM / 開發者 second brain	**Karpathy LLM Wiki**（or Anthropic Memory tool）	Obsidian + Mem0	—
AI Coding Agent 跨 session 記憶	**Anthropic Memory tool**（CLAUDE.md）	Supermemory / LLM Wiki	—
多 agent 共享 context	**Letta** + MCP	Mem0 + 自建 sync	—
已在 LangGraph 上的團隊	**LangMem**	—	—
ChatGPT C 端使用者	**OpenAI Memory**（內建）	—	不需 external 配置

關鍵 **reframe**：第三方 memory 框架（Mem0 / Zep / Letta）的市場空間，被 Long Context + Cache 從上方擠壓、被 Anthropic Memory tool 從側面擠壓，但在「self-host enterprise + 跨月跨年 + 多 agent autonomous」這三個 niche 仍是不可替代。選 **Mem0 / Zep** 之前，先問自己這三條路是否都不適用。

---

§8 失敗模式與 anti-pattern

> 本節結論
> - 13 種常見 anti-pattern 涵蓋：hallucination amplification、stale memory、cross-session contamination、cost runaway、cache 失效、GDPR 刪除權、資料主權
> - 最容易被低估的是 **cost runaway** 與 **GDPR / sovereignty** 風險——extraction-heavy 派每次 LLM call 累積，月成本可能超預期 10×
> - LLM Wiki 的 client-side 實作有 ~50% silent failure 率，需從 async background 改成 sync manual checkpoint
> - 中文場景預設 embedding 退化是常見坑，換 BGE-zh / jina-zh 可立即改善 12-18 點 recall

失敗模式	觸發場景	避免做法
Hallucination amplification	LLM extraction 寫入錯 fact，後續被多次強化	加 citation validator（rohitg00 v2 的補強）、定期 audit
Stale memory	Fact 演變但記憶系統未更新	用 Zep temporal model；或設 TTL
Cross-session contamination	多使用者共用 memory 但 scope 沒區分	嚴格區分 user_id / agent_id（Mem0 multi-scope）
Top-K gaming benchmark	為了好看的數字調 top_k 過大	評測時固定 top_k 為實際使用值
LLM Wiki client-side dropout	Background process 在 laptop 環境 timeout / silent fail	改 sync manual checkpoint（DEV.to 案例）
過度自動化掩蓋失敗	Karpathy 原 wiki 設計的 silent automation	「Simplicity First」——明確指令優於 silent automation
中文場景 **embedding** 退化	預設用英文 embedding 跑中文資料	換 BGE-zh / jina-zh
Schema 鎖定	LLM Wiki 一開始 schema 設定錯，後續難改	先做 minimal schema，邊用邊長
Fact 衝突無解	Mem0 / Zep 同主題抽出矛盾 fact	加 conflict resolution policy（社群已有實驗方案）
Cost runaway	Write-time LLM call × user 數 × session 頻率 → 月成本爆炸；Mem0 extraction-heavy 派最易發生	設 budget alarm；高頻 user 改用 verbatim / Long Context；批次 write 而非 per-message
Long Context cache 失效成本	每次 cache miss = 全 prompt 重算；超過 5min TTL 或 prompt 變動就 miss	sticky session；不要每 request 改 system prompt；長 prompt 拆成可獨立 cache 的 prefix + suffix
GDPR / 用戶刪除權無法落地	用戶要求刪除個人資料，但 fact 已散落 vector / KG / summary 多處；很難證明「徹底刪除」	設計時用統一 retention key；定期 audit；MemPalace 的 verbatim drawer + closet 反而較易 traceable
Hosted memory 的資料主權風險	Mem0 cloud / OpenAI Memory / Anthropic Memory tool 都把 user 資料存於第三方；regulated industry / sovereign 場景不可接受	預設用 self-host（Mem0 OSS / Cognee / MemPalace / Letta self-hosted）

LLM Wiki 在 client-side 的具體實作陷阱（DEV.to 案例細節）：

7 個專案 3 週實測，「about half the time, the background process either timed out, failed to parse the transcript, or exited silently with no output」。

三大失敗：
1. Background process dropout
2. Transcript parser choking on long sessions
3. Silent pipeline failures（hash check 對但 pipeline 沒跑）

修正方向：
- Async background → **Sync within active session**
- Transcript parse → **File-based date filtering**
- Silent automation → **Manual checkpoint command**（如 `/close-day`）
- 檔放 `.claude/` → 搬到 **project root**（`.claude/` 阻擋寫入）

> "Simplicity First. Minimum code that solves the problem"——引 Karpathy 自己的 principle 反向打他自己的初版設計。

---

§9 未來 12-24 個月應用想像

> 本節結論
> - 9 個方向分兩層：**(a)** 已有雛形可指（Cognitive Core / Personal AI / AI Coding / 科研 Autoresearch）；**(b)** 純推測有合理根據（多 agent Wiki / 教育 / 企業 KM / Robotics / Wiki→Fine-tune Loop）
> - 最直接落地：**Anthropic Haiku 4.5 + Memory tool + Files API** 已是商業可用 Cognitive Core 雛形
> - 最高賭注：**Wiki → Fine-tune Consolidation Loop**（把記憶烤回權重的循環）——若走通，當前所有框架都是過渡產品
> - 12 個月最可能落地：個人 PKM、AI Coding Agent 跨 session、企業 KM 取代 Confluence
>
> 本節分兩層：(a) 已有雛形可指；(b) 純推測但有合理根據。

§9.1 Cognitive Core 願景 — (a) 雛形

> "I want to remove the memory... and only maintain the algorithms for thought" — Karpathy

雛形（商業側為主、開源側為輔）：

- **Anthropic Haiku 4.5 + Memory tool + Files API**（最直接的商業可用 cognitive core 雛形）：Haiku 4.5 模型體量小、推論成本低、訓練後刻意保持「reasoning core，knowledge external」設計取向；配 Memory tool 持久化、配 Files API 做 retrieval，已是接近 Karpathy 概念的 production-grade 落地
- **OpenAI GPT-4.1 mini / nano + Responses API stateful threads**：類似路線，OpenAI 視角的 cognitive core 嘗試
- **Phi-3.5-mini（3.8B）+ 外掛 RAG**：開源 / 學術側雛形；edge 跑出近 GPT-3.5 表現
- **Llama 3.2 1B / Qwen2.5 1.5B**：開源小模型線正在強化「reasoning core，knowledge external」設計

12-24 個月推測：

- 商業側：Anthropic / OpenAI / Google 各自的 mini / nano / Haiku 線會與 memory primitive 深度整合，成為 edge agent / on-device agent 標配

- 開源側：~1B 參數的 cognitive core 模型線（Llama / Qwen / Gemma 後繼）會獨立成型，但與商業線的差距視 fine-tune 工具鏈成熟度
- 對 on-device agent（手機、車載、機器人）打開市場

§9.2 Personal AI / Second Brain — (a) 雛形

雛形：
- Obsidian + Smart Connections（vector search 插件）
- Notion AI、Mem.ai
- Karpathy LLM Wiki + agent assistant（Claude Code 已支援 CLAUDE.md 模式）

推測：
- 2027 前出現「個人 AI 平台」整合 Obsidian / Roam + Mem0 + LLM Wiki
- 個人 Wiki 變成跨工具一致的 source of truth
- 跨應用 context 透過 personal wiki API 同步（例：你跟 ChatGPT 講過的內容，Claude Code 也能讀）

§9.3 AI Coding Agent 跨 session 記憶 — (a) 雛形

雛形（直接接續本專案 `reports/2026-04-claude-code/` ）：
- Claude Code 的 `CLAUDE.md` 是手動版 LLM Wiki
- Cursor 的 `.cursorrules` 是手動版 procedural memory
- Supermemory + Claude Code 整合是自動版 Wiki

推測：
- IDE / agent 支援自動演化的 project knowledge wiki
- 跨專案 / 跨團隊的「coding pattern wiki」
- Pull request 時 agent 自動更新 architecture wiki
- 知識管理從「文件」升級到「agent 維護的活 KB」

§9.4 多 Agent 共享 Wiki — (b) 推測

> 為第二題（多 agent 編排研究）預留接口

目前缺：
- Mem0 / Zep / Letta 都假設單 agent / 單 user
- MCP shared context 還在早期
- Letta 的 memory blocks 是試水

推測架構：
- Wiki as service（HTTP API）
- 多 agent 透過 MCP read / write 同一 wiki
- Conflict resolution 透過 LLM 仲裁
- Audit trail 透過 git-tracked markdown

挑戰：concurrent write、stale read、agent 間信任邊界。

§9.5 教育 / Tutor agent — (b) 推測

架構：
- Wiki 記錄學生薄弱概念、學習偏好、進度
- 跨學期持續追蹤
- agent 自動調整教法（從 episodic learning event 反推 procedural teaching pattern）

雛形：Khanmigo（Khan Academy AI Tutor）已在試 cross-session memory，但仍弱在 procedural（記住「這個學生對視覺解釋反應好」）。

§9.6 企業 KM 取代 Confluence — (b) 推測

目前痛點：
- Confluence / Notion 內容不會自更新
- 員工不維護 → 文件腐化
- 搜尋體驗差

推測：
- Wiki maintained by agent，從 Slack / Email / 會議錄音 / git commit 自動更新
- 員工只負責「讀」+「修正錯誤」
- agent 每週 audit 內容一致性

挑戰：權限管理、隱私邊界、誤更新的回滾。

§9.7 Embodied / Robotics — (b) 推測

挑戰：
- 機器人需「跨任務技能累積」（學會開門 → 學會其他類似動作）
- 物理世界的 episodic memory 比文字密集得多
- Procedural memory（運動技能）難以用 markdown 表達

推測：
- Robotics-specific memory system（含 trajectory、視覺場景、力覺）
- 與 LLM-based memory 雙層架構：高層 LLM Wiki 記任務語義，低層 embedding 記 motor pattern

§9.8 科研 Autoresearch — (a) 雛形

雛形：karpathy/autoresearch——AI agents 自動跑單 GPU nanochat 訓練 + 實驗記錄。

推測：
- 跨實驗 wiki：記錄 ablation、超參、結果、假設修訂
- agent 自動 propose 下個實驗
- 文獻 review wiki + 實驗 wiki 雙向 cross-reference

對比現況：當前 ML 研究員手動維護 Notion / Notebook，效率瓶頸大。

§9.9 Wiki → Fine-tune Consolidation Loop — (b) 高賭注推測

> Karpathy 預告的 next step

架構：
1. Daily 對話 + raw sources
2. LLM 編譯成 wiki（episodic → semantic）
3. 累積一定量後，用 **wiki** 內容做 **personal fine-tune**（**QLoRA / LoRA**）
4. Fine-tune 完，原 wiki 部分內容可以從 prompt 中移除（已內隱權重）
5. 持續循環

這對應人腦的 **sleep consolidation**：白天經驗（episodic）→ 睡眠 hippocampus 重播 → cortex 長期儲存（semantic / implicit）。

為何是高賭注：
1. **Continual fine-tune** 的 **catastrophic forgetting** 仍未解 — 學新東西可能壞掉舊能力
2. **Personal model** 的經濟學：每個人需要自己的 fine-tune cycle，cost 不低
3. 隱私：fine-tuned weights 含個人記憶，等於數位人格的物質載體

潛在贏家：
- 個人化 fine-tune as a service（類比 1990s 個人電腦概念）
- "Personal Foundation Model"——你的個人 1B 模型，跟你一起學一輩子

風險：
- 模型能力不夠強，fine-tune 也撈不出來
- 巨頭壟斷（OpenAI / Anthropic 不釋放 weights）
- 隱私 / 安全災難（你的個人 weights 被偷 = 你的數位人格被偷）

如果這條路走通，當前 Mem0 / Zep / Letta / MemPalace 全部是過渡產品。真正的 **agent memory** 不是更好的外掛，是讓記憶部分回到模型本身。

---

§10 能力限制聲明

> 本節結論
> - 12 條限制中最關鍵的三條：**Karpathy-centric** 取材偏誤（七處引用、非 peer-reviewed）／**benchmark** 跨家口徑差異（不可直接比）／中文場景未實測（業界估計）
> - 主管 / 採購 / 投資判斷請以本聲明為前提，不要把報告當作「最終答案」
> - 這個領域當前最好的姿態是「持續關注 **+** 小型試點」，而不是「選定一家 all-in」

本研究存在以下無法克服的限制，撰寫時已盡量誠實標註：

1. 無法實測各框架的 **latency / API** 成本——所有性能數字引自官方 / 第三方公開資料，我們未自跑
2. **Karpathy** 觀點為個人視角——LLM Wiki gist 是非正式 GitHub gist 而非 peer-reviewed paper，社群仍在實證；System Prompt Learning 是 X 推文非完整論述；Cognitive Core 是 Dwarkesh podcast 訪談摘要。不應視為已驗證的工程藍圖
3. **LLM Wiki** 自維護的可靠性數據缺乏——目前僅有少數個案報告（包含 DEV.to 的批評文），無系統性 benchmark
4. **Benchmark** 數字可比性差——Mem0 / Zep / MemPalace 各家用不同 model + embedding + dataset 口徑，跨家直接比是不嚴謹的；本報告 §4 已盡量拆口徑，但仍可能誤導

5. **MemPalace 才剛釋出（2026-04-08）**——社群評測還在劇烈震盪，本報告引用的 88.9% 獨立重現是當前最可信數字，但未來 1-2 個月內可能再有大幅修正
6. 中文場景能力——所有主流框架都是英文中心，中文場景的真實表現本研究未實測，僅引用業界常見估計
7. 學術原型（**HiMem / SYNAPSE / TiMem / MAGMA**）的工程落地未驗證——這些 paper 給出概念貢獻但未必有可用 codebase，與生產級框架直接比較是不公平的
8. **§9** 應用想像分 **(a)(b)** 兩層：(a) 已有雛形可指、(b) 純推測但有合理根據；(b) 部分屬於趨勢判斷而非結論
9. 資料截止日 **2026-04-29**——agent memory 領域 2026 年迭代極快，本報告 1-2 個月後可能部分過時，特別是 MemPalace、LLM Wiki 社群、Letta MemFS 的新進展
10. **Karpathy-centric** 取材偏誤——本報告 §1.1 / §1.2 / §3.3 / §5.2 / §5.4 / §9.1 / §9.9 七處引用 Karpathy 的 LLM Wiki gist、System Prompt Learning X 推文、Cognitive Core podcast 訪談，作為「範式線」推論的主軸。這些都是非 **peer-reviewed**、非 **Anthropic / OpenAI / Google research roadmap** 的個人主張。Karpathy 是有號召力的研究者，他的觀點值得深思，但不應視為已驗證的工程藍圖。讀者應自行 weight：與當前主流 frontier lab（Anthropic / OpenAI / DeepMind）的官方研究路線對照，Karpathy 的賭注哪些被驗證、哪些仍在實證。報告已嘗試補入 Anthropic Memory tool / OpenAI Memory 等對照（§2.7-§2.8），但結構性偏誤無法完全消除
11. **Long Context + Cache** 路線的成本 / 效能數字未實測——§2.6 引用的 cache 折扣比例與 1M context 推論成本，均依官方文件估計；自部署開源模型 1M 落地的「品質衰退」「算力門檻」描述為業界常見估計，未自跑 benchmark
12. **Anthropic Memory tool** 的具體 **primitive** 規格仍在演進——§2.7 描述基於公開資料整理，實際 API 介面可能與本報告描述有出入；以 Anthropic 官方文件為準

主管 / 採購 / 投資判斷請以本聲明為前提，不要把報告當作「最終答案」。這個領域當前最好的姿態是「持續關注 **+** 小型試點」，而不是「選定一家 all-in」。

---

§11 參考資料

完整 URL 與訪問日期列在同層 `sources.md`。本節給出主要分類索引。

Karpathy 一手來源

- LLM Wiki gist（karpathy/llm-wiki，2026-04-04）
- System Prompt Learning X 推文（status 1921368644069765486，2025-05）
- Dwarkesh Patel podcast 訪談（2025-10）
- Latent.space S3 演講「Software 3.0」
- karpathy/autoresearch GitHub repo

Model Provider 官方資源

- Anthropic Memory tool docs（claude.ai/docs，§2.7 主要參考）
- Anthropic Files API + Claude Skills docs（§2.7）
- Anthropic Prompt Caching docs（§2.6）
- OpenAI Responses API + ChatGPT Memory blog（§2.8）
- Gemini Long Context + Context Caching docs（§2.6）

- LangChain LangMem module docs（§2.9）

框架官方資源

- Mem0 docs + State of AI Agent Memory 2026 blog
- Zep Knowledge Graph product page + Graphiti GitHub
- Letta docs + GitHub
- MemPalace GitHub（milla-jovovich/mempalace）+ 官方介紹
- Cognee docs

第三方對比

- Atlan：Best AI Agent Memory Frameworks 2026
- Letta forum：Letta vs Mem0 vs Zep vs Cognee
- DEV.to：Top 6 AI Agent Memory Frameworks for Devs 2026
- Graphlit：Survey of AI Agent Memory Frameworks

學術論文

- MemGPT（arXiv:2310.08560，2023-10）
- Mem0 LOCOMO（arXiv:2504.19413，ECAI 2025）
- ICLR 2026 MemAgents Workshop（OpenReview U51WxL382H）
- HiMem / SYNAPSE / TiMem / MAGMA / GAM / A-MEM / MIRIX 各論文

批評 / 修訂

- Nicholas Rhodes：MemPalace Review（2026-04）
- DEV.to：I Over-Engineered Karpathy's Agent Memory（2026-04-17）
- rohitg00：LLM Wiki v2 gist
- arXiv 2604.21284：Spatial Metaphors for LLM Memory（MemPalace 學術批評）

產業實踐

- LinkedIn Cognitive Memory Agent（ZenML LLMOps Database）
- VentureBeat：Karpathy LLM Knowledge Base 報導
- Memco / Gamgee / Epsilla / TecAdRise / MindStudio 商業化解讀

報告完。資料截止 2026-04-29。下一份相關研究：多 Agent 編排協議（MCP / A2A）與 shared memory 邊界。